BioSpecInfo

Matrice di tracciabilità — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v226** · commit `11a7ad7`

Generato il 2026-10-22

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Matrice di tracciabilità — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore	Samuele Pio Provenzano
Versione descritta	`bsi-v226`
Scopo	Collegare ogni requisito dichiarato all'implementazione che lo realizza e al banco di prova che lo verifica.

> **Come leggere questa matrice.** Ogni riga è una catena chiusa: un requisito, il punto del codice che lo realizza, e il banco che ne verifica il funzionamento. Una riga senza banco è un requisito **non verificato** e viene dichiarata come tale, non nascosta: l'utilità di una matrice sta esattamente nel rendere visibili le lacune.
>
> Gli esiti dei banchi citati sono riportati per esteso, con l'uscita integrale, in `evidence/RAPPORTO-VERIFICA.md`, generato automaticamente.

1. Requisiti scientifici

ID	Requisito	Implementazione	Banco di verifica
SCI-01	La struttura di un farmaco deve venire da una fonte esterna e coincidere con quella fonte, non solo pesare giusto	`FARM_DATA` in `index.html` — **263 voci**, 36 con identificativo ChEMBL e formula dichiarati	`audit_farmaci` — la formula viene **ricostruita contando gli atomi del grafo** e confrontata con quella di ChEMBL: 36 su 36. Il solo peso non bastava: tre voci preesistenti (lenalidomide, palbociclib, aripiprazolo) erano **isomeri** — stesso peso, molecola diversa — e sono state sostituite. Due cricche: provenienza verificata ≥ 36, strutture leggibili ≥ 240
SCI-02	I gruppi funzionali devono essere riconosciuti sulla struttura, non sul testo dello SMILES	`bsi-spettri.js` §1, pattern SMARTS via RDKit	`test_spettri` — 8 molecole di riferimento, riconoscimento additivo
SCI-03	Una molecola con più gruppi funzionali deve mostrarli tutti	`bandeDaGruppi()`, costruzione additiva	`test_spettri` §1 — acido acetilsalicilico: estere **e** acido

ID	Requisito	Implementazione	Banco di verifica
SCI-04	Gli spettri devono rispettare le convenzioni della tecnica	`svgIR()`, `makeNMRsvg()`, `drawUVSpectrum()`, `drawMSSpectrum()`	`test_assi` (SVG), `test_assi_canvas` (canvas)
SCI-05	Uno spettro calcolato deve essere riproducibile	rimozione di ogni `Math.random()` dalle tracce	`test_assi` §3, `test_spettri` §8 — confronto byte a byte
SCI-06	L'altezza dei picchi ^{1}H deve rappresentare l'integrazione o non essere asserita	`makeNMRsvg()`; `_nmrSvg()` disegna fasce a altezza uniforme	`test_assi` §4
SCI-07	La coniugazione deve agire sul singolo carbonile	SMARTS `acidoConiug`, `estereConiug`, `chetoneConiug`, `aldeideConiug`	`test_spettri` §1 — aspirina 1760/1690; acetofenone 1690
SCI-08	Una costante fisica ambigua non deve essere risolta arbitrariamente	`costante_fisica` in `bsi-ai-hub.js` — raccoglie tutte le candidate	`test_costanti` — 45 controlli, compresi i rifiuti per ambiguità
SCI-09	I limiti dei predittori devono essere dichiarati all'utente	intestazione dei grafici; `docs/06` §3.5	`verifica_guida` — coerenza fra promesse e codice
SCI-10	La simmetria molecolare deve determinare correttamente le regole di selezione	sezione `ssimm` in `index.html`	`test_simmetria` — 26 controlli: modi normali, centro di inversione, esclusione mutua
SCI-12	Un cromatogramma deve discendere dai parametri, non essere disegnato	sezione `scroma`: σ = t_R/√N, R_s da distanza e larghezze	`audit_stabilita` (apertura e disegno), ispezione dei valori contro le definizioni
SCI-13	Una retta di taratura deve dichiarare l'incertezza dei suoi coefficienti e riconoscere i due modi in cui inganna	sezione `staratura`: minimi quadrati ordinari, test F retta⟷parabola, residuo cancellato	Verificato sui tre insiemi di esempio inclusi: lineare (F = 0,3), non lineare (F = 186,6 > 7,71), con punto anomalo (1 rilevato)
SCI-14	Un modello QSAR deve essere valutato su scheletri mai visti, non su analoghi dello stesso insieme	`bsi-cheminfo.js` §9 — `divisionePerScaffold()` accanto a `divisioneCasuale()`	`test_cheminfo` — la divisione per scheletro non lascia nessuno scheletro in comune fra addestramento e prova
SCI-15	Un punteggio di modello deve essere confrontato con il caso, non presentato da solo	`bsi-cheminfo.js` §9 — riaddestramento su etichette rimescolate, otto ripetizioni	`test_cheminfo` — verificato **nei due versi**: su un segnale apprendibile R^2 0,394 batte tutti gli otto sosia (margine 0,287); su etichette casuali R^2 −0,199 **non** li batte

ID	Requisito	Implementazione	Banco di verifica
SCI-16	Due scritture della stessa molecola non devono contarsi due volte né finire da parti opposte della divisione	`bsi-cheminfo.js` §1 — deduplicazione sullo SMILES canonico dopo il frammento maggiore	`test_cheminfo` — `OC(=O)C` e `CC(=O)O` collassano in una voce; i sali perdono il controione
SCI-17	La similarità fra molecole deve essere calcolata, non stimata	`bsi-cheminfo.js` §3 — Tanimoto e Dice su impronte Morgan, RDKit, pattern, MACCS	`test_cheminfo` — confronto con valori calcolabili a mano, compresa la convenzione 0/0 = 1
SCI-18	Una validazione su una divisione sola, su un insieme piccolo, è un numero rumoroso: va accompagnata dalla sua dispersione	`bsi-cheminfo.js` §9-bis — `validazioneIncrociata()` su pieghe **raggruppate per scheletro**	`test_cheminfo` — le pieghe coprono ogni molecola una volta sola, **nessuno scheletro sta in due pieghe**, e con lo stesso seme le pieghe sono le stesse
SCI-19	Un risultato che cambia a ogni esecuzione non è verificabile da nessuno	generatore mulberry32 seminato per divisioni, rimescolamenti e pieghe; il seme è scritto nel rapporto	`test_cheminfo` — due chiamate con lo stesso seme danno pieghe identiche; il rapporto di metodo dichiara il seme
SCI-20	I risultati devono poter uscire dallo strumento, e il metodo deve uscire con loro	`bsi-cheminfo.js` §9-ter — `esportaCsv()` e `rapportoMetodo()`, entrambi formati in memoria nel browser	`test_cheminfo` — colonne coerenti con l'intestazione, nome con virgola citato, residuo calcolato, cella vuota per l'attività mancante; il rapporto contiene seme, fingerprint, verdetto, limiti
SCI-21	Un numero presentato come similarità strutturale deve venire da un fingerprint, e una sola implementazione deve calcolarlo	`rdkit_lab.html` carica `bsi-cheminfo.js` e ne usa `tanimoto()` e `fingerprint()`; la similarità a otto bit è stata rimossa	`test_cheminfo` — la pagina e il motore danno lo stesso Tanimoto su tutte e **28 le coppie**, scarto **esattamente 0**; le funzioni della similarità finta non esistono più; ogni struttura di riferimento è valida e ha la **massa monoisotopica** del farmaco che dichiara
SCI-22	Le conversioni fra linguaggi molecolari devono essere verificate contro una fonte esterna o contro il calcolo a mano, e le coordinate scritte in un file devono essere vere e dichiarate	`bsi-molingue.js` — **venticinque uscite**, da SMILES a CML e alle sei impronte	`test_lingue` — 7 chiavi InChI **di letteratura**, giro completo SMILES → molfile → SMILES, 26 nomi a mano, **7 rifiuti pretesi**; composizione in massa contro il calcolo a mano (alanina C 40,44%, aspirina C 60,00%); XYZ e PDB con coordinate non nulle **e** dichiarate 2D

ID	Requisito	Implementazione	Banco di verifica
SCI-23	Angoli, lunghezze e diedri si misurano sulle coordinate, e su coordinate piatte non si mostrano	prodotto scalare per gli angoli, prodotto vettoriale per i diedri	`test_mol3d` — tetraedro **109,4712°**, acqua 104,47°, CO_2 180°, BF_3 120°, ammoniaca 106,13°; diedri 0°, 60°, 90°, 180°; C–C dell'etanolo 1,509 Å; e il rifiuto provato nei due versi
SCI-24	Uno spettro letto da un file deve essere decodificato secondo il formato, e i picchi trovati devono essere quelli che ci sono — né più né meno	`bsi-spettrolettore.js` — JCAMP-DX con ASDF completo (PMAI, DIF, DUP) e il controllo di integrità che il formato prevede; picchi per **prominenza**, rumore da MAD × 1,4826	`test_spettrolettore` — **61 controlli**. Su tre gaussiane costruite a 1715, 2950, 3400 cm^{-1} sotto rumore ne trova tre: 1716, 2952, 3402. **Nei due versi**: su rumore puro ne trova **zero**. ASDF verificato simbolo per simbolo (`abcdefghi` = −1…−9; `n` = −5, non −4)
SCI-25	Una banda deve essere dichiarata *compatibile con*, mai assegnata a una sola possibilità	`BANDE_IR` — 24 intervalli, `assegnaIR()` restituisce **tutte** le assegnazioni compatibili	`test_spettrolettore` — a 1715 cm^{-1} compaiono sia C=O chetonico sia C=O di acido carbossilico; la sezione dichiara di non dedurre la struttura
SCI-26	Un modello pubblicato va verificato contro il **suo** errore dichiarato, e un modello appreso contro il caso	`bsi-cheminfo.js` §10 — ESOL (Delaney 2004) con incertezza, quattro filtri di drug-likeness separati, foresta casuale con errore fuori sacco	`test_spettrolettore` §4-5 — ESOL contro quattro valori sperimentali, scarto medio **0,68** entro l'errore dichiarato di ~1 unità logaritmica; foresta R^2 fuori sacco **0,90** su una relazione non lineare e **−0,16** su puro rumore; deterministica a parità di seme e diversa con seme diverso
SCI-27	Una struttura senza fonte non deve entrare, nemmeno quando la si conosce	raccolta da ChEMBL, `tools/dati/farmaci_v187.json` con l'identificativo di ogni voce	**Due rifiuti**: ivermectina (`structure_type NONE`, miscela di omologhi) e semaglutide (proteina, `SEQ`). E due nomi tenuti con il nome del **record** e non della domanda: «morphine» → apomorfina, «levothyroxine» → liotironina

ID	Requisito	Implementazione	Banco di verifica
SCI-28	Uno spettro previsto deve dire QUALI atomi producono ogni segnale, e lo scarto va misurato su molecole che non hanno scelto i parametri	`bsi-pretsch.js` — le tabelle di stima trascritte intere dalla fonte (Pretsch 4ª ed.: 91 righe ^{13}C e 66 ^{1}H per i benzeni, 42 per gli etileni, 31 per gli alcani, 24 per gli alifatici ^{13}C, correzioni steriche 4×4); `bsi-nmr.js` — il ragionamento che le applica: posizione nell'anello ricavata camminando il ciclo, schema additivo con composto di riferimento ciclico, equivalenza chimica per codice d'intorno a gusci	`test_nmr` — **56 controlli**. Due insiemi separati: taratura **0,71 ppm** su 9 molecole, **validazione 0,92 ppm** su 22 mai usate per tarare, caso peggiore **4,9 ppm**, ^{1}H **0,06 ppm** (0,03 sui soli aromatici). Il banco pretende che la validazione resti **peggiore** della taratura: se diventassero uguali, qualcuno avrebbe spostato una molecola fra i due insiemi
SCI-29	Atomi chimicamente equivalenti devono dare UN segnale, non uno per atomo	codice d'intorno a gusci concentrici per distanza (principio dei codici HOSE), con i legami aromatici scritti come tali e non in forma di Kekulé	`test_nmr` — otto molecole di cui si sa quanti segnali danno: benzene **1** (non sei), toluene 5, p-xilene 3, naftalene 3, difenile 4, aspirina 9. Tutte giuste
SCI-30	Da un'immagine si può recuperare la FORMA di uno spettro, non la sua taratura	`bsi-digitalizza.js` — `bsi-spettrolettore.js` gli delega `daImmagine()`, una sola implementazione. Cornice degli assi riconosciuta prima di tutto, traccia a **subpixel** (baricentro pesato sulla scurezza), scelta del tratto per continuità **e** per scurezza, taratura dei due assi lineare o logaritmica, %T convertita in assorbanza con $A = -\log_{10} T$	`test_digitalizza` — **45 controlli** su sette figure costruite: scarto medio **2,2 cm^{-1}**, peggiore **7,3** (una colonna vale ~4 cm^{-1}). **Nei due versi**: da un foglio bianco e dall'icona dell'applicazione non esce nessuna curva, e il ripiego grezzo sulla stessa figura con cornice ed etichette trova **un** picco invece di tre. `test_spettrolettore` per l'instradamento
SCI-31	Una mappa NMR bidimensionale deve dire QUALI atomi producono ogni correlazione, e non deve collegare ciò che la topologia non collega	`bsi-nmr2d.js` — HSQC a un legame con segno invertito sui CH_2, COSY a tre legami fra protoni non equivalenti, HMBC a due e tre legami; gli scambiabili esclusi dalle macchie fuori diagonale	`test_nmr2d` — **40 controlli**, nei due versi: l'esafluorobenzene non dà macchie HSQC, il benzene **nessuna** macchia COSY fuori diagonale (i sei protoni sono equivalenti), il metano **nessuna** macchia HMBC pur avendo quattro protoni
SCI-32	Una geometria 3D costruita dal programma deve avere GLI STESSI indici del predittore, altrimenti il collegamento picco↔atomo illumina l'atomo sbagliato	`bsi-geom3d.js` — geometria delle distanze sul grafo di `bsi-nmr.js`, idrogeni aggiunti in coda, generatore seminato dallo SMILES	`test_geom3d` — **29 controlli**: ogni atomo pesante ha lo stesso indice e lo stesso elemento su cinque molecole; legami e angoli entro tolleranza dai valori di letteratura; **nei due versi** il benzene esce piano e il cicloesano no

ID	Requisito	Implementazione	Banco di verifica
SCI-33	Un documento va aperto per intero, e ciò che NON si è potuto leggere va dichiarato — un lettore che restituisce poco senza dirlo fa credere che il documento fosse quasi vuoto	`bsi-documento.js` — PDF pagina per pagina con lo strato di testo (PDF.js), immagini, testo, archivi Office aperti con `DecompressionStream` senza librerie	`test_documento` — **47 controlli**: PDF, Word e testo aperti davvero e il loro contenuto verificato; **nei due versi** un'immagine dichiara che non c'è riconoscimento ottico dei caratteri e non produce svolgimento, un formato sconosciuto viene rifiutato invece che indovinato, un file binario chiamato `.txt` viene riconosciuto come binario
SCI-34	I dati spettroscopici riconosciuti in un testo devono portare la PROVENIENZA, e lo svolgimento deve dichiarare che cosa gli manca	`bsi-quesito.js` — riconoscimento per sezione (IR, MS, ^{1}H, ^{13}C) con i confini fra sezioni, poi `bsi-elucida.js` per lo svolgimento passo per passo	`test_documento` — formula, bande, masse con intensità, segnali ^{1}H con integrazione e molteplicità, segnali ^{13}C; **nei due versi**: $CDCl_3$ non diventa il composto, una banda fuori da 400-4000 non è una banda, senza etichetta MS non ci sono masse, un capoverso nuovo chiude la sezione, un testo senza spettri non produce dati
SCI-35	Una figura con la cornice, le etichette e la griglia non deve essere digitalizzata come se fosse solo la curva: gli assi e i numeri sono pixel scuri come la traccia	`bsi-digitalizza.js` — la cornice si cerca fra le linee CONTINUE nei margini, e un bordo si distingue da una riga di griglia per la **spaziatura irregolare** rispetto alla famiglia **oppure** per il tratto più spesso e più nero; il ritaglio si sposta oltre lo spessore della linea; le colonne di griglia verticale si segnano come non misurate e si ricostruiscono dalle vicine	`test_digitalizza` §1-2 — su una figura con cornice **e** griglia (11 famiglie verticali, 7 orizzontali) trova tutti e quattro i lati e i picchi restano entro 2,2 cm^{-1}; **nei due versi**, su una figura con la sola griglia **non** inventa una cornice e lo dichiara, e una **linea di base piatta** non viene presa per un asse
SCI-36	Una curva digitalizzata deve poter uscire dall'applicazione, e il file esportato deve essere valido	`bsi-digitalizza.js` — uscita CSV con le unità in testa e JCAMP-DX 4.24 in forma (XY..XY) con FIRSTX/LASTX/NPOINTS e la provenienza scritta dentro il file	`test_digitalizza` §4 — il **giro completo**: il JCAMP esportato viene riletto da `leggi()` di `bsi-spettrolettore.js` e torna con lo stesso numero di punti, gli stessi estremi e le stesse unità

ID	Requisito	Implementazione	Banco di verifica
UI-14	Una stessa funzione non deve avere due porte d'ingresso. Gli spettri stavano in **nove sezioni** su due menù diversi; Spectra in tre posti; e nel pannello ✨ comparivano **due «Data Science», due «File Manager», due «Articoli», due «Aggiornamenti»**, perché il pannello ha un elenco cablato che apriva le stesse sezioni inserite dal modulo del menù — e nessuno confrontava le due liste fra loro	`bsi-centrospettri.js` — i pannelli vengono **spostati** (`appendChild` muove, non copia) in una sola sezione a schede; i vecchi pulsanti restano nascosti perché sono il bersaglio di `goSection()`, e la scheda **clicca il pulsante** invece di mostrare il pannello, così l'inizializzazione pigra passa per l'unica strada che già c'era. `bsi-menu.js` — la categoria «Utility» è nel **menù principale** con tutte le sue voci, e la regola è: se una cosa ha un pulsante **raggiungibile** nella barra non sta nel menù ✨, se non ce l'ha sta solo nel menù. «Raggiungibile» non si misura con `offsetParent` — nella barra un gruppo per volta è mostrato, e un pulsante perfettamente raggiungibile ha `offsetParent` nullo	`test_articoli` §5-6 — le **dieci** schede sono dentro il centro, e ogni identificatore esiste nel documento **una volta sola** (la prova che si è spostato e non copiato); `goSection()` su ognuna apre il centro sulla scheda giusta e ne mostra **solo** una. `test_menu` — Utility visibile nella barra con tutte e **quindici** le voci più le due delle lingue, **nessuna** voce del menù che ripeta una sezione della barra, **nessuna** etichetta doppia nel pannello, e su **due viewport**: schermo grande e telefono. **Nei due versi**: nessun id ripetuto, nessun pulsante doppio, Spectra e il lettore fuori dalla barra ma con il pulsante ancora nel documento perché `goSection()` lo usa
UI-15	Una tabella di dati non deve esistere due volte: due copie divergono, e chi legge ottiene una risposta diversa a seconda di dove entra	Le costanti di accoppiamento J esistevano in due tabelle indipendenti con numeri **diversi** (^{3}J orto aromatico 6–9 Hz contro 7–10 Hz). Resta quella dentro il Centro spettri, che è nella sezione con la sua lista di riferimenti; `_apriCostantiJ()` è l'unica strada e porta lì	`test_articoli` §6 — chiedere le costanti J apre il centro sulla scheda delle tabelle di riferimento, sulla sotto-scheda giusta, e la tabella ha le sue righe
ING-02	Un elenco di articoli scritto a mano è vecchio il giorno dopo, e un anno di premio scritto a memoria è un errore che non si vede	`bsi-articoli.js` legge **in diretta** da Crossref e dalla Fondazione Nobel; `bsi-articoli-nobel.js` contiene **solo spiegazioni**, abbinate all'argomento della motivazione ufficiale e non all'anno, così nessuna attribuzione viene dall'applicazione. Gli indirizzi non sono scritti: sono **funzioni pure** dell'identificatore (DOI, anno)	`test_articoli` §1-4 — i link provati come funzioni (stabili a parità di dato, diversi al variare); le spiegazioni abbinate a nove motivazioni di prova e **nessuna** inventata quando l'argomento non si riconosce; e la prova che conta: si **inietta** una risposta con un nome inesistente e si pretende che sullo schermo compaia **quel** nome. **Tre guasti distinti provati**: richiesta fallita, risposta vuota, risposta di forma non riconosciuta — e devono essere detti **diversi**

ID	Requisito	Implementazione	Banco di verifica
UI-16	Cambiare lingua deve costare un gesto, non una navigazione	`bsi-sellingua.js` — un pulsante **tondo** in alto a destra, all'altezza del titolo, con la bandiera in uso; si apre in un disco con le quattordici bandiere su **due gusci** (sei dentro, come un anello benzenico) e al centro il nome della lingua e **quanto è tradotta, misurata** da `coperturaPerLingua()`. Il cambio vero resta in `bsi-lingue.js`: qui c'è solo il modo di chiamarlo, e la sezione «Lingua» resta perché INFORMA mentre questo AGISCE	`test_sellingua` su **due viewport** — tondo, in alto, a destra, e «all'altezza del titolo» **misurata** come sovrapposizione dei due riquadri (con `align-self:center` usciva allineato su schermo grande e no su telefono, perché là il flex allinea in alto e il blocco del logo è alto due righe); nome accessibile che dice che cosa fa e non «bandiera dell'Italia»; il disco **dentro lo schermo**; e **nei due versi**: la lingua cambia davvero e tornando all'italiano il testo è identico a prima
ING-01	Una stessa funzione non deve avere due implementazioni: lo stack React/FastAPI deve eseguire gli STESSI file dell'applicazione	`stack/api/worker/motore.mjs` — un processo Node carica i moduli dal repository con RDKit WebAssembly; nessun motore in Python	`test_stack` — **15 controlli**: confronta le impronte SHA-256 dichiarate da `GET /salute` con quelle dei file su disco, verifica che i numeri coincidano con quelli del browser, e che nessun file `.ts` / `.tsx` contenga una tabella di spostamenti

2. Requisiti dell'agente AI

ID	Requisito	Implementazione	Banco di verifica
AI-01	Un modello ritirato dal fornitore non deve bloccare l'assistente	`modelloSuggeritoDaErrore()`, lista nera `bsi_modelli_ko`	`test_404` — 36 controlli
AI-02	Un fornitore irraggiungibile dal browser deve essere ricordato e retrocesso	`bsi_prov_ko`, TTL 24 h, `providerUtilizzabili()`	`test_ko` (35), `browser_ko` (37)
AI-03	Un sovraccarico temporaneo deve produrre un nuovo tentativo, non un errore	`erroreTemporaneo()`, `TENTATIVI_TEMPORANEO`	`test_503` — 40 controlli
AI-04	Un'attesa imposta superiore a 10 s deve far cambiare fornitore se ve n'è uno pronto	`ATTESA_TROPPO_LUNGA_MS`	`test_attesalunga` — 14 controlli
AI-05	Il tetto di token del fornitore deve essere appreso dall'errore, non codificato	`tettoDaErrore()`, `MARGINE_TETTO`	`test_tetto` — 32 controlli

ID	Requisito	Implementazione	Banco di verifica
AI-06	Le firme di ragionamento opache devono essere restituite al fornitore	cattura e replay in `appendAgentTurn()`	`test_firma` — 31 controlli
AI-07	Due invii sovrapposti non devono corrompere la cronologia	`_turnoInCorso` in `send()`	`test_doppioinvio` — 7 controlli
AI-08	L'utente deve poter misurare quali fornitori raggiunge davvero	`provaTuttiIFornitori()`	`browser_prova` — 29 controlli
AI-09	I fornitori a pagamento non devono essere proposti come gratuiti	separazione per `PROVIDERS[id].free`	`browser_prova` §D2
AI-10	Deve esistere una via documentata per eliminare CORS e limiti	`proxy/spectra-proxy.js` + guida in-app	`test_guidaproxy` (12), `browser_proxy` (15), `browser_proxyui` (17)

3. Requisiti di stabilità

ID	Requisito	Implementazione	Banco di verifica
STA-01	Una sessione lunga non deve accumulare nodi DOM né timer	gestione del ciclo di vita delle sezioni	`audit_stabilita` §1 — 95 sezioni × 5 giri
STA-02	L'esaurimento di `localStorage` non deve rendere inutilizzabile l'app	scritture protette; avviso persistente nel File Manager	`audit_quota` (10), `test_filemanager` (15)
STA-03	Nessuna promessa rifiutata deve restare non gestita	`.catch()` sistematici	`audit_promesse` — 22 controlli, rete sana e rete morta
STA-04	Dati salvati corrotti non devono impedire l'avvio	`loadJSON()` con ripiego	`audit_stabilita` §4
STA-05	Una rete lenta non deve essere trattata come una rete assente	`ATTESA_RETE_MS = 3500` in `sw.js`	`test_sw` §3 e §3-bis — misurato 3 507 ms
STA-06	Un aggiornamento non deve distruggere il lavoro in corso	messaggio `BSI_SW_UPDATED`, decisione lato pagina	`test_sw` §4
STA-07	Sfogliare molecole non deve esaurire i contesti WebGL	riuso del visore 3D nei 5 punti di creazione	`test_visore3d` — da 7 costruzioni a 1
STA-08	L'archivio IndexedDB non disponibile deve essere dichiarato, non fatale	`dbStore()` su promessa; avviso persistente	`test_filemanager` §2

4. Requisiti di interfaccia e accessibilità

ID	Requisito	Implementazione	Banco di verifica
UI-01	Ogni pagina deve aprirsi senza errori JavaScript	—	`audit_stabilita` §5 — 14 pagine
UI-02	I grafici devono essere nitidi su schermi ad alta densità	`bsiNitido()` sui contesti canvas	`audit_grafici` — 40 canvas
UI-03	L'app deve funzionare a 390 px di larghezza, e il contenuto che non ci sta deve restare **raggiungibile**	griglie `auto-fit`, nessuna colonna fissa, e le tabelle larghe dentro un contenitore con `overflow-x:auto`	`audit_mobile` — **due misure distinte** su tutte le 95 sezioni a 390 px: lo `scrollWidth` del documento (la barra orizzontale che l'utente vede) **e** gli elementi più larghi della loro sezione **senza un antenato scorrevole**, cioè il contenuto ritagliato e irraggiungibile — una colonna scritta che non si può leggere e non dà nessun segno di esistere. **Nei due versi**: una tabella larga dentro un contenitore che scorre **non** viene segnalata, perché si legge scorrendola. `audit_stabilita` per gli errori in viewport telefono
UI-04	L'utente deve poter sapere quale versione sta usando e forzare l'aggiornamento	voce «Aggiornamenti» nel pannello ✨	`test_aggiorna` — 9 controlli
UI-05	La cancellazione dei dati deve essere selettiva e reversibile nelle scelte	`bsiCancellaDati()` per gruppi	`browser_reset` — 24 controlli
UI-08	Anche le superfici che compaiono solo dopo un'azione devono rispettare il contrasto WCAG AA	i sei pannelli della sezione Chemioinformatica	`test_cheminfo` — l'analisi viene eseguita, i pannelli aperti a turno, **460 elementi di testo misurati, 0 difetti**, con le formule WCAG riscritte dentro il banco

ID	Requisito	Implementazione	Banco di verifica
UI-09	Aprire una sezione non deve bloccare la pagina	primo `render()` di 3Dmol fuori dal click; le 296 figure di sintesi **e le 263 carte dei farmaci** disegnate a fette da 8 ms	`test_fluidita` — **95 sezioni attraversate**, cambio mediano 16 ms, peggiore 87 ms, **0 sezioni oltre 100 ms** (erano 3 con i farmaci nuovi, a 159 ms); e le prove contrarie: la tela WebGL compare comunque, le 296 figure esistono tutte poco dopo, la stampa non esce muta. Sul visore l'affermazione è **strutturale e non temporale**: subito dopo il ritorno del gestore del click il contesto WebGL non deve esistere, poco dopo deve — perché una soglia in millisecondi misurava SwiftShader e non il codice (14…192 ms a codice identico)
UI-10	L'interfaccia deve poter cambiare lingua fra molte, dichiarando per ognuna quanto è tradotta, portando con sé il verso di scrittura	`bsi-lingue.js` con **quattordici lingue**, selezionatore con ricerca, `dir="rtl"` per l'arabo	`test_lingue` — **170 elementi di scheletro su 170 in ognuna delle tredici lingue**, copertura calcolata applicando il dizionario e contando; verso di scrittura provato nei due versi; categorie della barra e segnaposto della ricerca compresi
UI-11	La molecola deve mostrare come sta insieme, e l'angolo lo deve scegliere chi guarda	`bsi-mol3d.js`: formazione dalla polvere con arrivo scaglionato, e selezione degli atomi con un clic	`test_mol3d` — i **pixel** della tela a 130 ms e a 2,3 s, l'ordine d'arrivo (scheletro prima degli idrogeni) misurato sull'avanzamento di ogni atomo, e la selezione di 1, 2, 3 e 4 atomi
UI-12	Chi ha un file deve poterlo aprire: un selettore che elenca le estensioni non restringe la scelta su un telefono, la **nasconde**	`bsi-spettrolettore.js` — un solo `<input type="file">` **senza `accept`**, un solo bottone, e `apriQualunque()` che instrada per tipo, per estensione dichiarata dal formato e **per contenuto**, annunciando la decisione in `#bsiSP-deciso`	`test_spettrolettore` §7 — un solo bottone visibile, `accept` **vuoto** (un `accept` non vuoto qui *è* il guasto), e l'instradamento verificato su PDF, `.docx`, `.txt`, `.jdx`, due colonne `.csv` e un'immagine; **nei due versi** un binario travestito da `.txt` non viene riversato come testo ma dichiarato binario
UI-13	La versione mostrata in intestazione deve essere quella che sta girando	`index.html` — il distintivo si riempie da `window.BSI_APP_VERSION`, non è più una stringa scritta a mano	`test_spettrolettore` §8 — il testo del distintivo è confrontato con `BSI_APP_VERSION` misurata nella pagina in esecuzione. Serviva: per circa ottanta versioni l'intestazione ha scritto `v13l` quando l'applicazione aveva già superato la versione 190, e nessun controllo la guardava

5. Requisiti di sicurezza e riservatezza

ID	Requisito	Implementazione	Banco di verifica
SEC-01	Nessuna chiave API deve essere presente nel repository	chiavi solo in `localStorage` o nei segreti del Worker	`tools/verifica-sicurezza.js` — 371 file tracciati, 8 forme di credenziale
SEC-02	Nessun dato personale deve lasciare il dispositivo senza azione esplicita	architettura local-first, telemetria disattivata	`audit_rete` — **verifica diretta**: un valore spia seminato in 71 depositi dei dati utente, l'applicazione usata per 95 sezioni su 6 pagine, e URL, intestazioni e corpo di ogni richiesta ispezionati. Più `verifica-sicurezza` sui due meccanismi di uscita
SEC-03	Le password non devono comparire in chiaro nel sorgente	SHA-256 in `file_manager.html`	`tools/verifica-sicurezza.js`
SEC-07	Nessuna credenziale deve essere MAI entrata nel repository, nemmeno in un commit poi corretto	nessuna chiave è mai stata committata; le chiavi stanno in `localStorage` o nei segreti del Worker	`audit_storia` — **1 551 versioni distinte di file di testo su 390 commit**, 9 schemi provati nei due versi; fallisce su un clone superficiale, perché misurerebbe meno superficie
SEC-04	Nessun marcatore di conflitto deve raggiungere la pubblicazione	—	`verifica_guida` §12 — 58 file di testo

> **SEC-03 — limite dichiarato.** GitHub Pages serve file statici: qualunque controllo di accesso lato pagina è aggirabile da chi legge il sorgente. La protezione del File Manager è un deterrente, **non** un controllo di sicurezza, e la documentazione lo afferma. Inoltre la password in chiaro è rimasta nella cronologia git fino alla sua rimozione, e togliere un segreto dai file non lo toglie dalla storia: `git log -p` lo restituisce a chiunque. L'unico rimedio effettivo era cambiarla, ed **è stato fatto** alla versione `bsi-v188`. Quella vecchia resta nella cronologia e non apre più niente.

---

6. Requisiti non ancora verificati automaticamente

Dichiarati per completezza. Sono coperti da ispezione documentale o da prova manuale, e la loro automazione è in programma.

ID	Requisito	Copertura attuale
UI-06	Conformità WCAG 2.1 AA completa	la parte meccanica è automatizzata su **13 pagine e 95 sezioni** (`tools/verifica-accessibilita.js`, **38 407** elementi di testo: erano 19 751 prima che il testo su gradiente entrasse nella misura, e 37 409 prima della sezione «Lettore spettri»). **Dei 1 069 difetti di contrasto emersi non ne resta nessuno: 0 misurati** sulle stesse 95 sezioni, e il valore è registrato come riferimento che il banco difende. I campi senza etichetta sono **zero**: vedi il riquadro in `docs/09` §4. Restano fuori il testo dentro gli SVG e quello su una vera immagine di sfondo, contati a ogni esecuzione (D-09)
PERF-01	Tempo di primo disegno su dispositivo di fascia bassa	prova manuale cross-device (`docs/02` §4)
SCI-11	Strutture di 2 voci che non sono molecole singole (erano 6)	**non rappresentabili**: Ivermectina è una miscela di omologhi, Coartem un'associazione di due principi attivi. Le altre quattro sono state chiuse riprendendo la struttura da ChEMBL, vedi `docs/06` §2.4
PERF-02	Copertura di codice dei banchi di prova	**misurata parzialmente**: 49,89 % di istruzioni sul percorso più ampio (`audit_copertura`). Resta non misurata la copertura dell'intera batteria e quella di rami; vedi §8
UI-07	Comportamento su Firefox e WebKit	**non verificato** — la batteria gira solo su Chromium; vedi §8

7. Copertura complessiva

Categoria	Requisiti	Verificati da banco	Copertura
Scientifici (SCI-01…36)	35	35	100 %
Agente AI (AI-01…10)	10	10	100 %
Stabilità (STA-01…08)	8	8	100 %
Interfaccia (UI-01…16)	14	14	100 %
Sicurezza (SEC-01…07)	5	5	100 %
Architettura (ING-01…02)	2	2	100 %
Totale automatizzato	**74**	**74**	**100 %**
Non automatizzati (§6)	5	0	0 %
Totale dichiarato	**79**	**74**	**94 %**

La copertura è calcolata sui requisiti **dichiarati in questo documento** e non va confusa con una copertura di codice: misura quanti requisiti hanno un banco che li verifica, non quante righe vengono eseguite durante i test.

> **Perché due totali.** Riportare solo il 100 % dei requisiti automatizzati sarebbe vero e fuorviante insieme: è il 100 % di ciò che si è scelto di automatizzare. Il numero che conta per chi valuta è il secondo — **54 requisiti verificati su 59 dichiarati** — e i cinque che mancano sono elencati per nome in §6, non riassunti in una percentuale.
>
> Questa tabella è controllata da `tools/verifica-documenti.js`, che conta gli identificativi realmente presenti nel documento e fallisce se i totali qui scritti non corrispondono. È un controllo nato da un errore reale: i totali dicevano 36 su 9 categorie scientifiche mentre gli identificativi definiti erano 10, e `SCI-10` compariva due volte — una come requisito verificato e una come requisito non verificato.

---

8. Lacune di copertura dichiarate

Non sono difetti: sono cose che **non sono state misurate**, e che quindi non possono essere affermate.

Lacuna	Che cosa comporta	Perché è così
Copertura di codice parziale	Misurata: **49,89 %** di istruzioni sul percorso più ampio che un banco compie. Non è la copertura dell'intera batteria, ed è di istruzioni, non di rami	Il profilatore di Chromium la raccoglie nel motore, senza build e senza riscrivere il sorgente: l'ostacolo era dello strumento (c8, istanbul), non del problema. Valore registrato, `audit_copertura` fallisce se scende
Solo Chromium	Il comportamento su Firefox e WebKit è verificato a mano, non da banco	La batteria usa `playwright-core`, che scarica un motore solo. Nell'ambiente di verifica la CDN degli altri motori risponde **403** alla politica di rete: non sono installabili lì
Nessuna regressione visiva	Un cambiamento grafico involontario non verrebbe intercettato	Gli spettri sono deterministici e confrontabili byte a byte (SCI-05): il confronto esiste sulle tracce, non sull'intera pagina
2 voci senza struttura	Due voci su 178 non hanno una struttura da mostrare: Ivermectina è una miscela di omologhi, Coartem un'associazione di due principi attivi. Erano sei	Vedi `06-Scientific-Accuracy-Data-Provenance.md` §2.4
Prestazioni su dispositivi lenti	Il tempo di primo disegno non è misurato su hardware di fascia bassa	Richiede dispositivi fisici; la prova è manuale

---

Documento aggiornato alla versione `bsi-v226`.